Databases and SQL

DATA201/422 Data Wrangling

John Bryant

2026-09-08

Normalisation

Removing redundancy

Before

order	product	units	cost per unit
1	Apples	2	$5
2	Bread	1	$6
3	Bread	3	$6
4	Apples	1	$5
5	Carrots	4	$2

After

order	product	units
1	Apples	2
2	Bread	1
3	Bread	3
4	Apples	1
5	Carrots	4

product	cost per unit
Apples	$5
Bread	$6
Carrots	$2

Another example

Before

order_id	product	units	type	customer
101	Apples	2	Fruit	Ana
101	Bread	1	Carbs	Ana
102	Bread	3	Carbs	Ben
102	Carrots	4	Veg	Ben

After

order_items

order_id	product	units
101	Apples	2
101	Bread	1
102	Bread	3
102	Carrots	4

products

product	type
Apples	Fruit
Bread	Carbs
Carrots	Veg

orders

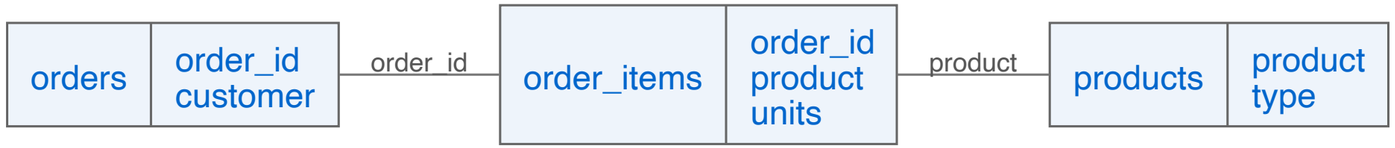
order_id	customer
101	Ana
102	Ben

Theory of normalisation is surprisingly complicated

- “First normal form”, “second normal form”, etc
- Details usually not important for data science
  - More important for database designers, administrators
- But basic understanding *is* very useful

Advantages and disadvantages of normalising

- Advantages
  - Can clarify system being modelled



  - Easier to extend, validate data
  - More efficient storage
- Disadvantages
  - Can be harder to construct queries (lots of joins)
  - Can have lots of trivial tables

Modular SQL

SQL queries can easily get very long

Prompt: “Please create a typical example of a long SQL query”

```sql
WITH customer_sales AS (
  SELECT
    c.customer_id,
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS number_of_orders,
    SUM(oi.quantity * oi.unit_price) AS total_revenue
  FROM customers AS c
  INNER JOIN orders AS o
    ON o.customer_id = c.customer_id
  INNER JOIN order_items AS oi
    ON oi.order_id = o.order_id
  WHERE o.order_date >= DATE '2025-01-01'
    AND o.status IN ('completed', 'shipped')
  GROUP BY c.customer_id, c.customer_name
  HAVING COUNT(DISTINCT o.order_id) >= 3
)
SELECT
  customer_id,
  customer_name,
  number_of_orders,
  ROUND(total_revenue, 2) AS total_revenue,
  DENSE_RANK() OVER (
    ORDER BY total_revenue DESC
  ) AS revenue_rank
FROM customer_sales
WHERE total_revenue > 1000
ORDER BY revenue_rank;
```

- Long statements are hard to understand, validate, or debug
- Better to break into pieces

ASIDE

Modularity

- Key idea in computer science
  - Maybe the best idea in all of computer science
- Definition
  - Lots of small components
  - Each component has one clearly defined job
  - Each component makes minimal assumptions about other components
  - Information passed between components in clear, explicit way
- Benefits
  - Easier to understand (small, digestible pieces)
  - Flexible, reusable - recombine in different ways
  - Easier to fix if something goes wrong
- Classic example: functions

Database views

- In SQL database, a ‘stored query’
  - Refer to the view as if was a table, and the stored query runs
  - Only the *query* is stored, not the data itself
- Creating views does not duplicate data
  - Uses minimal storage
  - Preserves Single Source of True

Creating a database view

Data

products

id	name
1	Apple
2	Bread
3	Pear
4	Milk
5	Eggs

ratings

name	rating
Apple	yum
Bread	yuck
Milk	yuck
Pear	yum

SQL

```
1  CREATE VIEW product_rating AS
2  SELECT
3    products.id,
4    products.name,
5    ratings.rating
6  FROM products
7  INNER JOIN ratings
8  ON products.name
9    = ratings.name
```

View

id	name	rating
3	Pear	yum
1	Apple	yum
4	Milk	yuck
2	Bread	yuck

Using a database view

View

product_rating

id	name	rating
3	Pear	yum
1	Apple	yum
4	Milk	yuck
2	Bread	yuck

SQL

```
1 SELECT id, name
2 FROM product_rating
3 WHERE rating = 'yuck'
```

Results

id	name
4	Milk
2	Bread

Effect on performance

- Depends on database, query
- Even if there is a performance penalty, that may be OK if safety and clarity improve

Temporary vs permanent views

- Permanent view
  - Persistent part of the database
  - Appropriate for standard, frequently used parts of workflow
  - May require approval from database administrator
- Temporary view
  - Add the key word ‘TEMPORARY’

```
1   CREATE TEMPORARY VIEW product_rating AS
```

  - Only lasts for the current session
  - Typically does not require special permissions
  - Lives in scripts (owned by data scientist) not database

Common Table Expressions (CTEs)

- Like a view, but defined within the statement itself

```sql
WITH product_rating AS (
  SELECT
    products.id,
    products.name,
    ratings.rating
  FROM products
  INNER JOIN ratings
  ON products.name
    = ratings.name
)
SELECT id, name
FROM product_rating
WHERE rating = 'yuck'
```

Comparison of CTEs and views

- Both achieve modularity
- Advantages of views
  - Hides detail better
  - Easier to test
- Advantages of CTEs
  - Less administration
  - Easier to change as project evolves

AI and SQL

Coding agents can be really good at SQL

- Lots of code for them to learn on
- Lots of literature on best practice
- AIs have bigger working memories than humans
  - Can cope with large SQL statements

Implications for data science?

- Strengthens case for using databases within pipeline
  - Simple transformations to large datasets
- Job of data scientist is to design specifications and validate
  - Less emphasis on writing detailed code

Good practice

- Learn to read SQL
  - Not that hard; AI can help
- Include lots of checks, assertions
  - Check for unique rows, agreement with known totals, etc
  - Boring for humans to code, but AI agents don’t mind
- Maintain natural-language description as part of project?
  - Get the agent to describe the data processing in plain English
  - Base the description entirely on the SQL and database schemas
  - Check that it matches expectations, and update as project develops
- Be careful with letting AI see sensitive data
  - Check agent’s data handling policies
  - Consider working with de-identified or simulated data